

Laboratorio di Programmazione

Esercizi di esame — livello medio (08, 09, 10)

Esercizio 08 — Rubrica telefonica da file CSV

Un file CSV contiene una rubrica con colonne **nome**, **cognome**, **telefono**, **email**. Crea la classe `PhoneBookFile`, inizializzata col nome del file, con un metodo `get_contacts()` che restituisce la lista dei contatti come dizionari. Crea poi la classe `PhoneBook`, inizializzata con la lista di contatti, con tre metodi: `search(query)` che cerca per nome o cognome (ricerca parziale, case-insensitive); `get_duplicates()` che individua contatti con lo stesso numero di telefono; e `export_sorted(field)` che restituisce la lista dei contatti ordinata per il campo indicato.

ESEMPIO D'USO

```
pb_file = PhoneBookFile(name='rubrica.csv')
contacts = pb_file.get_contacts()

pb = PhoneBook(contacts)
results = pb.search("ros")
# [{"nome": "Mario", "cognome": "Rossi", ...}, ...]
dups = pb.get_duplicates()
# [{"telefono": "333111222", "contatti": ["Mario Rossi", "Anna Rossi"]}, ...]
sorted_list = pb.export_sorted("cognome")
# lista ordinata alfabeticamente per cognome
```

SPECIFICHE

Classe `PhoneBookFile`

- Il file viene aperto solo in `get_contacts()`; se non esiste o non è leggibile, alzare `ExamException`.
- Righe con campi mancanti o con numero di telefono non composto solo da cifre vanno ignorate (stampare un messaggio).
- L'email non è obbligatoria: se assente, il campo vale `None`.
- Se il file non contiene almeno un contatto valido, alzare `ExamException`.

Metodo `search(query)`

- Se `query` non è una stringa, alzare `ExamException`.
- Se `query` è una stringa vuota, alzare `ExamException`.
- Se nessun contatto corrisponde, restituire una lista vuota.

Metodo `get_duplicates()`

- Due contatti sono duplicati se hanno lo stesso numero di telefono (confronto esatto).
- Se non ci sono duplicati, restituire una lista vuota.

Metodo `export_sorted(field)`

- Se `field` non è uno tra "nome", "cognome", "telefono", alzare `ExamException`.
- L'ordinamento deve essere case-insensitive.

- Non modificare la lista originale: restituire sempre una nuova lista.

Esercizio 09 — Analisi voti da file di testo

Un file di testo contiene i voti degli studenti, uno per riga, nel formato `matricola, nome, voto`. Crea la classe `GradeFile` con metodo `get_grades()` che restituisce la lista dei record come dizionari. Crea poi la funzione `compute_statistics(grades)` che ritorna un dizionario con le statistiche globali, e la funzione `compute_grade_distribution(grades)` che raggruppa i voti per fascia (18–21, 22–25, 26–28, 29–30, 30L).

ESEMPIO D'USO

```
gf = GradeFile(name='voti.txt')
grades = gf.get_grades()

stats = compute_statistics(grades)
# {"media": 25.3, "mediana": 26.0, "min": 18, "max": 30,
#  "promossi": 42, "bocciati": 5}

dist = compute_grade_distribution(grades)
# {"18-21": 8, "22-25": 12, "26-28": 14, "29-30": 6, "30L": 2}
```

SPECIFICHE

Classe `GradeFile`

- Il file viene aperto solo in `get_grades()`; se non esiste o non è leggibile, alzare `ExamException`.
- I voti validi sono interi da 18 a 30, oppure la stringa "30L". Qualsiasi altro valore va ignorato (stampare un messaggio).
- Righe con campi mancanti o con matricola non numerica vanno ignorate.
- Una stessa matricola non può apparire due volte: in caso di duplicato, ignorare la seconda occorrenza e stampare un messaggio.
- Se il file non contiene almeno un record valido, alzare `ExamException`.

Funzione `compute_statistics(grades)`

- Se `grades` è una lista vuota, alzare `ExamException`.
- Per il calcolo della mediana: con numero pari di elementi, usare la media dei due centrali.
- Ai fini delle statistiche numeriche, "30L" vale 30.
- Il campo "promossi" conta tutti i record validi; "bocciati" conta le righe ignorate per voto non valido (non le righe malformate).

Funzione `compute_grade_distribution(grades)`

- Restituire sempre tutte e cinque le fasce come chiavi, anche se il conteggio è zero.
- "30L" va conteggiato nella fascia "30L", non in "29-30".

Esercizio 10 — Contatore di sequenze in una time series

Data una time series numerica (stessa struttura dell'esercitazione 2), crea la classe `SequenceFinder`, inizializzata con i dati, con due metodi:

`find_longest_streak(direction)` che trova la sequenza consecutiva più lunga di valori in aumento ("up") o in diminuzione ("down"); e `find_threshold_crossings(threshold)` che individua tutti i punti in cui il valore supera (da sotto a sopra) o scende sotto (da sopra a sotto) una soglia data.

ESEMPIO D'USO

```
from esercitazione2 import CSVTimeSeriesFile

tsf = CSVTimeSeriesFile(name='data.csv')
ts = tsf.get_data()

sf = SequenceFinder(ts)
streak = sf.find_longest_streak("up")
# {"start": "1949-06", "end": "1949-10",
#  "length": 5, "values": [135, 148, 152, 160, 172]}

crossings = sf.find_threshold_crossings(200)
# [{"date": "1951-05", "direction": "up", "value": 204},
#  {"date": "1952-01", "direction": "down", "value": 193}, ...]
```

SPECIFICHE

Costruttore

- Se i dati passati non sono una lista di liste o la lista è vuota, alzare `ExamException`.
- Se la lista contiene meno di 2 elementi validi, alzare `ExamException`.

Metodo `find_longest_streak(direction)`

- Se `direction` non è "up" o "down", alzare `ExamException`.
- Una sequenza "up" è una serie di valori strettamente crescenti; "down" strettamente decrescenti.
- La lunghezza minima di una sequenza è 2. Se non esiste alcuna sequenza valida, alzare `ExamException`.
- In caso di più sequenze della stessa lunghezza massima, restituire la prima in ordine temporale.
- Il dizionario di output include: "start" (data primo elemento), "end" (data ultimo), "length" (numero di elementi), "values" (lista dei valori).

Metodo `find_threshold_crossings(threshold)`

- Se `threshold` non è un numero, alzare `ExamException`.
- Un crossing "up" avviene quando il valore al passo $t-1$ è $\leq$ `threshold` e al passo t è $>$ `threshold`. Un crossing "down" è il contrario.
- Se non ci sono crossing, restituire una lista vuota.
- Ogni elemento della lista di output è un dizionario con "date", "direction" e "value" (il valore al momento del crossing).

In tutti gli esercizi, le eccezioni vanno alzate come istanze di `ExamException(Exception)`. Chiamate il file `esame_matricola.py`. Commentate il codice. Non usare librerie esterne né il modulo `datetime`.